

SWEN-342 Engineering Of Concurrent & Distributed Software Systems

The Woolies

Overview

This activity has **three** incremental deliverables. As you complete each section, submit the specified file(s) into the designated dropbox.

Part 1

In the land of the Woolies, the towns of Merctran and Sicstine are on opposite sides of the Zyxnine River. There is a single bridge that connects these two towns. Any Woolie that wishes to go to the town on the other side of the river must cross this bridge.



In this activity you will create a class, named **Woolie**, that simulates a Woolie crossing the bridge. The bridge can carry more than one Woolie at a time, so you will have to write the **Woolie** class so that each simulated Woolie runs in a separate thread.

Every Woolie has a name, and like most people, Woolies walk at different speeds. The constructor of the **Woolie** class will be given: the Woolie's name, the number of seconds it takes the Woolie to cross the bridge, and the name of the city (Merctan or Sicstine) that the Woolie is going to. The **run()** method of the **Woolie** class simulates the Woolie crossing the bridge. The Woolie starts crossing the bridge as soon as the **run()** method begins execution. The **run()** method consists of a loop, that executes once for every second the Woolie is on the bridge.

There are several message that must be displayed to describe the Woolie's progress crossing the bridge. *Note: In all the following messages name is the name of the*

Woolie.

- When the Woolie thread starts, the following message is display:

`name has arrived at the bridge.`

- When the Woolie starts crossing the bridge, at time 0, the following message is display:

`name is starting to cross.`

- For every one second interval, beyond time 0, that the Woolie is on the bridge a message

`"tab" name x seconds.`

should be printed where x is the number of seconds that the Woolie has been on the bridge and "tab" is the tab character - '\t'.

- When the Woolie reaches his or her destination display the message

`name leaves at city.`

where `city` is the Woolie's destination.

Test your **Woolie** class by writing a test program that creates multiple Woolies with different names, crossing times and destinations. Be sure to verify the following things:

- multiple Woolies can be on the bridge at the same time,
- each Woolie is on the bridge for the proper length of time, and
- each Woolie exits at the correct destination.

Note that for this activity you ***do not*** have to create a class to simulate the bridge. You only need to write the **Woolie** class, and then write a test program that creates a number of Woolies and prints out what they are doing.

Deliverables (Part 1):

Submit your **Woolie.java** file to the Activity 3-1 dropbox in myCourses.

Part 2

The bridge itself is very old and the municipal engineers have noticed many cracks developing in the structural members of the bridge. The state of the bridge is so

bad that the engineers feel that they need to limit the bridge to carry only one person at a time. The Woolies are a rather impatient and unruly bunch of people. Even though they know that the bridge might collapse under the weight of more than one Woolie, plunging them into the Zyxnine River, they will not limit crossing to one at a time.



The government has hired a mean-spirited troll to police the bridge and limit passage to a maximum of one Woolie at a time.

In this section you will write a **Bridge** class that simulates the bridge troll who controls passage across the bridge. Each Woolie will need to get permission from the bridge troll before he or she can cross the bridge. The bridge troll should make sure that only one Woolie is on the bridge at a time. When a Woolie crossing the bridge gets to the other side, the bridge troll will be notified so that another waiting Woolie can be allowed on.

The **Bridge** class contains two methods: `enterBridge()` that a Woolie invokes to indicate that they wish to cross the bridge, and `leaveBridge()` that is invoked when the Woolie has finished crossing the bridge. The `enterBridge()` method will contain the code that ensures that only one Woolie is on the bridge at a time. The `leaveBridge()` method contains the code that notifies the troll that the Woolie is off the bridge. Remember that Java synchronization mechanisms can be used to do most of the hard work of the **Bridge** class.

You will also have to modify the constructor and the `run()` method of the **Woolie** class. The modified constructor will take a fourth parameter that provides a reference to the bridge that the Woolie wishes to cross. This additional parameter should be the last parameter to the constructor. This reference should be saved as an additional instance variable in the **Woolie** class. The modified `run()` method will need to get the permission of the troll before proceeding across the bridge, by

invoking `enterBridge()`, and will need to notify the bridge troll, using `leaveBridge()`, after the bridge has been crossed. The output from Part 1 generated by the `run()` method will not change.

Deliverables (Part 2):

Submit your **Woolie.java** and **Bridge.java** files to the Activity 3-2 dropbox in myCourses.

Part 3

As you know the bridge between Merctran and Sicstine is very busy. Now with only one pedestrian allowed on the bridge at a time there are long lines of Woolies waiting to cross the bridge. The municipal engineers decided to analyze the bridge and make the necessary fixes so that it can support up to 3 Woolies at a time.



The Woolies who were using the bridge were also getting upset because the bridge troll did not keep track of the order in which the Woolies arrived at the bridge. When the time came to allow another Woolie to cross the bridge, the bridge troll was selecting a Woolie at random, and granting them permission to cross the bridge. The Woolies made it clear to their government representatives, that the troll should be instructed to let the Woolies cross in the order in which they arrived at the bridge.

In this activity you will change the **Bridge** class so that the bridge troll allows up to three Woolies on the bridge at one time, and that the Woolies cross the bridge in the order in which they arrive. The bridge troll will need to maintain a collection of Woolies waiting to cross the bridge and be sure to give them permission to cross in the order in which they were added to the collection. Since each Woolie executes in a separate thread of execution, the troll should keep a collection of **Woolie** threads. A Woolie will announce his or her arrival at the bridge by invoking the `enterBridge()` method of the **Bridge** class, and will notify the troll that they have

finished crossing the bridge by invoking the `leaveBridge()` method. When a Woolie arrives at the bridge and can not immediately begin crossing, its thread must enter the wait state. When there is space on the bridge the bridge troll will notify the Woolies who are waiting to cross the bridge. You must ensure that only the Woolie who is next actually starts crossing the bridge, the other Woolies must continue waiting.

Deliverables (Part 3):

Submit your **Bridge.java** file to the Activity 3-3 dropbox in myCourses.